

# PixelSurvey: An open-source framework for creating surveys with YAML

Francisco Garrido-Valenzuela<sup>\*1</sup>, Oded Cats<sup>2</sup>, and Sander van Cranenburgh<sup>1</sup>

<sup>1</sup>CityAI Lab, Transport and Logistics Group, Delft University of Technology, the Netherlands

<sup>2</sup>Department of Transport & Planning, Delft University of Technology, the Netherlands

## ABSTRACT

Surveys are among the most widely used tools in behavioral, social, and psychological research. At the same time, visual information is increasingly recognized as central to how humans perceive, evaluate, and decide, underscoring the need for survey infrastructures that systematically and reproducibly integrate images. However, existing platforms face significant limitations when it comes to incorporating image-based experimental designs. PixelSurvey, an open-source, modular framework for designing, deploying, and sharing survey experiments addresses this gap. Built around human-readable YAML “recipes” that fully specify survey structure, logic, and stimuli, PixelSurvey allows the inclusion of image-based experimental designs. Additionally, it ensures transparency, reproducibility, and ease of sharing. Its standardized architecture supports different question types and image-based experiment paradigms, while its modular design enables community-driven extensions. By lowering technical barriers and embedding open-science principles, PixelSurvey provides a versatile and sustainable platform for advancing the study of human perceptions, preferences, and decision-making.

**Keywords:** image-based survey, stated choice experiment, similarity judgment, web-platform, python-dash

## 1 INTRODUCTION

Surveys are among the most widely used methodological tools in behavioral research. They are commonly employed to investigate human preferences, perceptions, and decision-making processes across a broad range of domains. These include transportation (Ortuzar et al., 2000), urban studies (Salm et al., 2023), medicine (Green & Gerard, 2009), and marketing (Palma et al., 2018). Surveys are well-suited for capturing subjective judgments, eliciting stated choices, and assessing attitudes under controlled experimental conditions. The existence of dedicated research fields such as stated choice modeling (Louviere et al., 2000; Mariel et al., 2021), conjoint analysis (Hainmueller et al., 2014; Al-Omari et al., 2022), factorial survey experiments (Forster & Neugebauer, 2024; Steiner et al., 2016), contingent valuation with dichotomous choice designs (Hanemann et al., 1991), and experimental design (Rose & Bliemer, 2014) attests to the relevance that surveys play in the empirical study of behavior. Beyond academic contexts, surveys are extensively used in industry for a variety of purposes, including consumer sentiment analysis, user experience reporting, political polling, and market forecasting. As such, surveys have become an important component in both the scientific investigation and the practical application of behavioral understanding.

A wide variety of software and platforms are currently available for building and managing surveys. Broadly, these can be divided into general-purpose platforms and research-specialized ones. General-purpose platforms (e.g., SurveyMonkey (2025); LimeSurvey (2025)) are widely used in academia, industry, and government because they are robust, user-friendly, and versatile. They allow users to design questionnaires, deploy them online, and collect responses. These platforms typically support standard question types such as multiple-choice, Likert-scales, rankings, and open-ended items. Research-specialized platforms (e.g., Qualtrics (2025); SurveyEngine (2025); Chen et al. (2016)) go one step further by offering finer control over survey logic, randomization, and experimental design, which makes them particularly suited for behavioral and social science studies. Within this category, psychological and cognitive research often relies on code-based libraries such as jsPsych (De Leeuw, 2015), PsychoPy (Peirce et al., 2019) or lab.js Henninger et al. (2022). These frameworks provide maximal flexibility in designing stimulus-based tasks

but some of them demand programming expertise. All these platforms provide a rich ecosystem of options that cater to different levels of expertise, study requirements, and methodological traditions across fields such as psychology, sociology, transportation, and marketing.

Alongside this diversity of tools, a growing body of research highlights the central role of visual information in human behavior and decision-making. People naturally rely on visual cues to evaluate environments, products, and experiences (Paivio, 1969; Pinker, 1990). Some recent studies show that image-based designs can also reduce social desirability bias and increase validity in behavioral experiments, particularly in fields such as sociology and political science (Vecchiato & Munger, 2025; López Ortega & Radojevic, 2025). As a result, image-based approaches are increasingly used in experimental tasks to assess perceptions (Hebart et al., 2020; Garrido-Valenzuela et al., 2025) or preferences (Rizzi et al., 2012; van Cranenburgh & Garrido-Valenzuela, 2025) and their potential relevance extends to areas such as travel behavior, environmental economics, and psychology. At the same time, recent advances in computer vision have made it increasingly feasible to analyze, process, and incorporate imagery into behavioral research designs. Deep learning models (He et al., 2016; Liu et al., 2022; Dosovitskiy et al., 2020) support classification, embedding, and retrieval at scale, allowing researchers to incorporate large and diverse visual datasets without extensive manual pre-processing. Therefore, computer vision developments offer new opportunities to incorporate imagery into behavioral research designs, especially in contexts where appearance or visual context are relevant to decision-making or perceptions (van Cranenburgh & Garrido-Valenzuela, 2025). As a last point, image-based surveys have been found to be more engaging and enjoyable for participants compared to text-based formats (PIHappiness, 2025; SurveyLegend, 2025). This not only sustains attention and reduces fatigue but also tends to increase completion rates, thereby lowering the effort required to recruit respondents and achieve target sample sizes.

However, despite the growing relevance of images in surveys, most existing platforms provide limited support for managing experimental designs that rely on large-scale image datasets. On the one hand, some platforms do allow the inclusion of static images as part of survey content. For instance, in LimeSurvey (2025), PIHappiness (2025), and SurveyLegend (2025), images can be used as alternatives in multiple choice tasks or as stimuli in Likert-scale questions. Yet these tools generally do not allow for a systematic control over image-based attributes, randomization of visual alternatives across respondents, or the combination of images with textual or numerical variables in unified designs Schwitter (2025). This limitation becomes particularly relevant when images represent explanatory variables and must be sampled, grouped, or presented under specific design constraints. In such cases, researchers are often required to manually process and organize image files, prepare custom randomization scripts, and maintain separate linkages between images, tasks, and responses. On the other hand, research-specialized platforms such as Qualtrics (2025) and SurveyEngine (2025) offer sophisticated experimental logic but are not natively (i.e., without coding) equipped to handle image-based tasks at scale. As a result, deploying image-based survey experiments remains technically challenging, particularly for non-programmers or interdisciplinary teams without dedicated computational support.

Beyond the handling of images, current platforms also remain limited in other relevant aspects. To date, no open-source framework exists that provides a complete environment for designing, deploying, and analyzing survey with experiments. The most similar platform is LimeSurvey (2025), which is open source, but does not allow one to create experiments guided by experimental designs. This limits transparency, reproducibility, and opportunities for collective improvement. In light of broader concerns about the reproducibility crisis and the growing call for open science practices (Nosek et al., 2015; Munafò et al., 2017), the absence of open and shareable survey infrastructures is particularly problematic. Lack of transparency not only hampers replication but also increases the risk of questionable research practices or even fraud in survey-based studies (John et al., 2012). Furthermore, most platforms treat surveys as isolated projects that cannot easily be shared in full—including their structure, logic, and stimuli—for replication or reuse. Finally, survey elements are rarely designed as modular components that can be exchanged or improved collaboratively. This prevents the development of community-driven libraries of reusable modules, such as standardized question types or experimental designs, that could accelerate innovation and foster comparability across studies.

To address this omission in the toolkit of behavioral researchers, we introduce PixelSurvey, an open-source, modular framework for designing and deploying survey experiments. PixelSurvey directly responds to the challenges outlined above. First, it provides a general infrastructure for experimental surveys, where tasks can be flexibly defined through reusable components such as questionnaires and multiple experiment types. Second, it offers native support for image-based

experiments, enabling systematic control over large image databases, randomization of visual alternatives, and integration of images with textual or numerical variables in unified designs. Third, it is fully open-source and organized around human-readable YAML "recipes" files. YAML files are simple structured-files that encapsulate the survey structure, logic, and stimuli in a single item. This facilitates sharing, replication, transparency, and adaptation across studies. Finally, its component-based architecture promotes modularity and community-driven extensions, allowing researchers to contribute new question types, experimental paradigms, or designs. Taken together, PixelSurvey lowers the technical barrier to implementing complex survey experiments while extending support for visually rich research designs.

This paper introduces PixelSurvey in detail. We begin with an overview of current platforms and their capabilities and limitations for surveys with experiments. We then present the architecture and workflow of PixelSurvey. This section explains the framework, survey structures, and how surveys are generated from human-readable YAML recipes into fully functional web applications. Additionally, we describe the core components of the framework, including survey sections, activity types, and the supporting database. Specifically, this article introduces three different image-based experiment types which are currently available through PixelSurvey. We conclude with examples of its usage that illustrate how to prepare and build a survey, and discuss opportunities for community contributions.

## 2 EXISTING TOOLS

A variety of software platforms exist for constructing surveys and experimental tasks, each of which with its own distinct strengths and limitations.

### 2.1 *General-purpose platforms*

General-purpose survey platforms, such as LimeSurvey and SurveyMonkey, provide robust infrastructures for questionnaire design, branching logic, basic randomization, and large-scale data collection. These tools are widely adopted in both academic and commercial research due to their reliability and user-friendly interfaces. However, they are primarily designed for conventional surveys with text- or number-based input, and their support for experimental image stimuli is limited. Images can be inserted as static content within questions, and there is no built-in mechanism to systematically manipulate large sets of images or integrate them into complex experimental designs. Additionally, implementing specialized sampling, or advanced randomization schemes is often difficult or impossible, within these environments. Although LimeSurvey is open source and accepts community contributions, it is not intended for experimental design in behavioral research.

### 2.2 *Research-specialized platforms*

Research-specialized survey platforms have been developed to support more complex experimental designs than conventional questionnaire tools, and are widely used across fields such as psychology, medicine, sociology, political science, and transportation. SurveyEngine (2025) is particularly oriented toward stated choice experiments and provides dedicated modules for constructing choice sets and managing experimental designs. However, its support for image stimuli is limited, and it does not allow systematic control over large sets of images within structured experimental frameworks. In sociology and political science, other platforms such as SoSci Survey (Leiner, 2025) and Unipark (Questback GmbH) (Questback GmbH, 2025) are commonly used to implement vignette and factorial survey experiments, often with complex randomization schemes. But, they face similar limitations in handling large-scale image datasets. Qualtrics (2025), in contrast, offers a more flexible environment where researchers can integrate images and extend functionality through custom JavaScript code. This enables the implementation of experimental designs with images. However this comes at the cost of introducing a technical barrier for researchers without programming expertise. Overall, all these platforms are proprietary, which restricts transparency and sharing of complete survey designs. Their commercial nature also means that access can be costly, limiting their adoption for smaller projects or in academic settings where open and reproducible tools are preferred.

For studies that require fine-grained control over stimulus presentation and timing, researchers

often turn to specialized experimental software. Tools such as PsychoPy (Peirce et al., 2019) or jsPsych (De Leeuw, 2015) allow precise control of stimulus timing, flexible trial configuration, and the implementation of advanced experimental paradigms. These frameworks are central in psychology and cognitive science, where millisecond accuracy and complex stimulus logic are essential. They typically run as desktop applications or JavaScript libraries and can present stimuli in controlled sequences with rigorous timing. Other open-source frameworks such as oTree (Chen et al., 2016) and lab.js (Henninger et al., 2022) have become increasingly popular in behavioral economics, psychology, and sociology for building interactive or vignette-based survey experiments online. While all of these tools expand flexibility and support reproducible experimental designs, they are not primarily designed to integrate survey structures with large-scale image stimuli. Additionally, some of them (i.e., PsychoPy and jsPsych), generally requires scripting or programming skills.

### 2.3 Statement of need

Researchers who wish to study human preferences and perceptions systematically through experiments (with images) face a clear trade-off. On one hand, general-purpose survey platforms offer accessibility and scalability but lack the flexibility needed for experimental designs and large-scale image management. On the other hand, research-oriented frameworks deliver experimental rigor but require considerable programming expertise and are not designed for integrating survey structures with image stimuli at scale. As a result, current tools either compromise ease of use or methodological power, leaving researchers without an accessible solution that supports both large-scale image handling and experimental control. PixelSurvey addresses this need by providing an open-source, modular framework that integrates user-friendly survey functionality with fine-grained experimental design. It enables researchers to incorporate large image datasets, systematically manipulate and randomize them, and combine them with textual or numerical variables within unified designs. In doing so, PixelSurvey lowers the technical barrier associated with implementing complex survey experiments, expands the methodological toolkit available for studying human perception and decision-making, and facilitates open science practices.

## 3 PIXELSURVEY

PixelSurvey is an open-source, modular, and community-oriented framework for creating and deploying image-based surveys. Its development is guided by four core principles. First, it is fully open source, ensuring transparency, reproducibility, and enabling community-driven contributions. Second, it adopts a modular architecture, in which survey elements are defined as independent components that simplify future extensions and can be flexibly modified. Third, it is designed to be user-friendly, lowering the technical barrier for researchers who wish to implement visually rich survey experiments without extensive programming expertise. Finally, PixelSurvey is designed to be community-based. We aim to foster collaboration, sharing survey templates, and collective improvement of the platform. Together, these principles make PixelSurvey a unique and versatile framework for collecting human perceptions and preferences. While PixelSurvey’s primary motivation was to enable systematic management of image-based stimuli, its flexible architecture now extends far beyond images, making it equally suitable for experiments built entirely on textual, numerical, or other data inputs. In the following subsections, we describe its architecture, workflow, main functionalities, and illustrative use cases.

### 3.1 Architecture

The central component of PixelSurvey is **PixelSurveyCore**, which provides the main functionality of the framework. PixelSurveyCore receives a *survey recipe* as input and, based on this configuration, automatically generates a web application. Figure 1 illustrates this process, in which a *survey recipe* is used to generate the web service with its own database to collect responses.

The survey recipe consists of two elements: a YAML file (i.e., recipe file) and a source folder (both detailed later). The recipe file specifies the survey structure, experiments, and settings; and the source folder contains all additional files referenced in the YAML such as experimental designs or logos. This architecture ensures that surveys are represented in a single YAML file, making

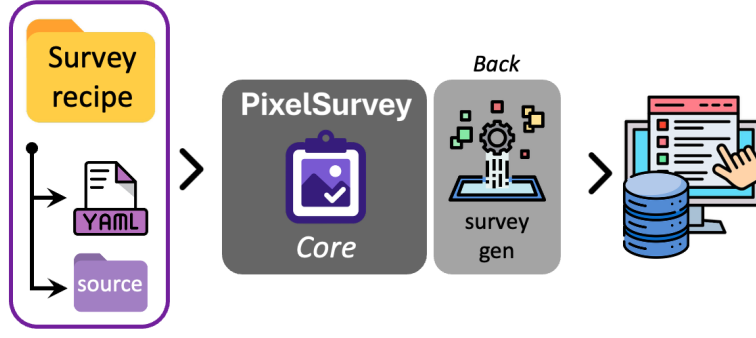


Figure 1: General PixelSurvey pipeline.

them clearly specified, easy to share, and reproducible across different configurations.

### 3.2 A survey in PixelSurvey

PixelSurvey adopts a standardized survey structure that is shared across all implementations. This design reduces the complexity of building surveys. Each survey generated with this framework is organized into three sections: On-boarding, Body, and Closure. This structure provides a modular template that improves the readability of survey recipes. In other words, the sections act as an organizational (not functional) framework that facilitates survey design and comprehension, but the actual logic of the experiment is defined at a more granular level (i.e., pages).

Each section is composed of one or more pages, which represent the actual functional units of a survey. These pages correspond to individual web pages encountered by participants and may differ in purpose and content. Although this structure allows for flexibility, it also introduces a certain degree of rigidity to ensure coherence and standardization. Figure 2 shows an overview of the sections and their pages, and the following subsections describe the role of each section in the survey.

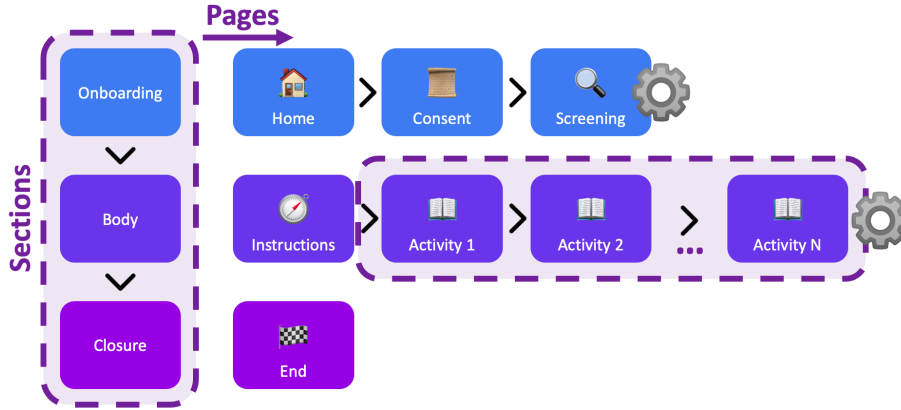


Figure 2: General sections and pages' structure of a survey in PixelSurvey. Blocks with gear icons indicate components that include questions/experimental content and can be customized by users.

#### 3.2.1 On-boarding section

The on-boarding section always contains three pages: Home, Consent, and Screening. These pages are intended to introduce participants to the study, ensure ethical and legal compliance, and confirm eligibility. The Home page typically presents a welcome message and general information about the survey. The Consent page provides the informed consent statement, outlining participant rights and data protection policies, and requires explicit agreement before proceeding. Finally, the Screening page allows researchers to include questions (e.g., demographics, prior experience) to determine whether participants meet predefined inclusion criteria or to proceed with a representative sample of a target population.

While users can freely modify the content of all survey pages, the Screening page has a special role. It allows researchers to include any number of multiple-choice questions that can be used to filter participants and facilitate data collection through demographic quotas or sampling strategies that aim to represent a larger population. This flexibility allows the Screening page supports questions customization and enables targeted participant recruitment.

### 3.2.2 Body section

The body section constitutes the central part of the survey. It always begins with an Instructions page, which provides participants with task-specific guidance and clarifies the expected responses. This is followed by one or more activities, which can be either questionnaires or experiments. Figure 3 shows the current components for each of those activities.

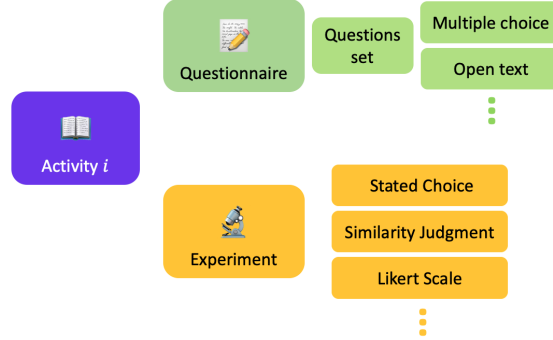


Figure 3: Activities available in PixelSurvey: Experiments (Stated choice, similarity judgment, and Likert scale) and Questionnaires. Community can contribute with new experiments and questions types

Questionnaires and experiments are the two activity categories available in PixelSurvey. Within a single survey, researchers may combine multiple activities to meet their design requirements. Questionnaires consist of a sequence of one or more questions, which may include multiple-choice type, where respondents select among predefined alternatives, or open-text type, where participants provide free-text answers. Experiments, in contrast, are designed to present structured tasks based on stimuli and controlled conditions. At present, PixelSurvey implements three types of experiments: Stated Choice, Similarity Judgment, and Likert Scale (each described in detail in the following subsections). While Likert scale items are conventionally treated as a question type, in PixelSurvey they are implemented as an experiment because they can be systematically repeated across sets of images or instances, enabling controlled collection of perceptual evaluations. Although the current version provides only two question types and three experiment types, the framework has been designed to be extensible. The community can contribute additional question and experiment types, which can be integrated into future releases of the tool. For instance, people can contribute with a simple Likert question type—i.e., a single rating item without experimental repetition.

### 3.2.3 Closure section

The closure section contains a single page, the End page, which finalizes the survey. Its main purpose is to acknowledge the participant’s contribution and provide optional debriefing information. The End page may also include a thank-you message, contact information for further questions, or a redirection link (e.g., to a participant pool system or follow-up study).

## 3.3 The survey recipe

As introduced, the survey recipe is the configuration unit that defines how a web survey instance is built. It is organized as a folder containing two main elements: a YAML file (the recipe file) and a source folder. These components describe the structure of the survey and provide all the resources needed for deployment.

A YAML file is a human-readable configuration document that encodes structural information in plain text. Instead of requiring users to write code, the YAML file organizes information in a

hierarchical format based on key–value pairs and indentation. This makes it intuitive to read and edit, even for researchers without programming expertise. These files are also widely used in other projects such as Home Assistant, Docker, or Conda, where they serve as a simple way to define system configurations, device settings, or software environments.

### 3.3.1 General YAML structure

The survey YAML encodes the survey’s sections and pages, the activities to be presented (e.g., questionnaires or experiments), and the parameters that control their behavior, such as experimental designs. Because it is written in a human-readable format, the YAML file allows researchers to directly inspect, modify, and share experimental designs without requiring programming skills. This makes it possible to reproduce a survey exactly, or to adapt it by editing only a few lines of configuration. The survey YAML has the following main structure:

```
survey:
  name: "My Survey"

  banner:
    title: "Title"
    subtitle: "Subtitle"
    logo: source/logo.png
    color: "#00A6D6"

  sections:
    onboarding:
      home:
        ...
      consent:
        ...
      screening:
        ...
    body:
      instructions:
        ...
      activity_1:
        ...
      activity_2:
        ...
      activity_N:
        ...
    closure:
      end:
        ...
```

The top-level key **survey** contains all information associated with a survey and is subdivided into **name**, **banner**, and **sections** properties. The **name** field specifies the project name. The **banner** field defines the settings for the top banner of the web application, including the **title**, **subtitle**, **logo**, and **color** scheme. Finally, the **sections** field declares the three mandatory components of every survey: **onboarding**, **body**, and **closure**, each containing the respective pages introduced earlier. Figure 4 illustrates the general layout of a survey page using the YAML banner example shown above.

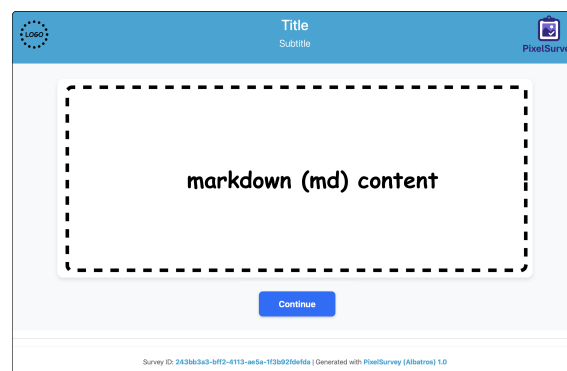


Figure 4: Page layout (web) of a page in PixelSurvey

Survey pages share a common layout consisting of four elements: a banner at the top, a content area, a navigation button, and a footer. The banner displays the survey title, subtitle, the user-defined logo, and the PixelSurvey logo. The main content area is usually written in Markdown format<sup>1</sup>, but it is a dynamic space that adapts to the type of page (e.g., instructions, questions, or experiments). Markdown allows for structured text, including headings, emphasis, or lists, while remaining lightweight and easy to edit. A navigation button is always included to proceed to the next page (except for the End page). Finally, the footer closes the page. Specific page settings are described in detail in the following subsections.

### 3.3.2 *On-boarding/home*

The Home page welcomes respondents to the survey and provides introductory information about the study. It is defined in the YAML recipe with a single property, **content**, which stores text written in Markdown format. In YAML, the `"|"` symbol indicates that all subsequent indented lines should be interpreted as Markdown content. This YAML snippet shows a Home page example:

```
home:
  content: |
    ## Survey example
    Welcome to our survey example. In this survey you have to ...
```

### 3.3.3 *On-boarding/consent*

The Consent page is structurally similar to the Home page but is designed to meet ethical requirements for online research. It contains a **content** property for displaying the consent text in Markdown format, and an additional **statement** property that defines the exact text participants must agree to before proceeding. This YAML snippet shows a Consent page example:

```
consent:
  content: |
    ## Informed Consent
    I would like to invite you to participate ...
    As with any online activity, ...

  statement: "I consent to participate in this study"
```

### 3.3.4 *On-boarding/screening*

The Screening page is the first point at which researchers can introduce customized questions for participants. Its purpose is to assess eligibility and, if needed, to control the composition of the sample (e.g., demographics or quota-based recruitment). The YAML definition includes the **content** property which is Markdown text to provide guidance, **questions** property to define one or more multiple-choice questions, and **quotas** to instruct the platform how to handle quota limits based on the screening question responses. This YAML snippet shows a Screening page example:

---

<sup>1</sup>Markdown is a lightweight markup language that uses plain text formatting syntax, allowing for headings, emphasis, lists, and hyperlinks. It is widely used in platforms such as GitHub, Jupyter Notebooks, and many documentation systems because it is both human-readable and easy to render into HTML or PDF.



```

screening:
  content: |
    ## Participant Screening
    Please answer the following questions ...

  questions:
    - id: 1
      ...
    - id: 2
      ...

  quotas:
    ...

```

The `questions` property contains a list of individual questions, each following a standard template described later in the section on survey components. These questions are multiple-choice and can be used to filter participants according to study requirements. The `quotas` property determines how responses to the screening questions are used to regulate sample composition. This allows researchers to impose limits on all possible multiple-choice alternative combinations. PixelSurvey supports three methods of quota management: *no\_limit*, *uniform*, and *custom*. Figure 5 shows the YAML snippets for the three different configuration for quotas.

```

quotas:
  method: no_limit
  fullquota_content: |
    # Thank you ...
    Unfortunately, you ...

```

(a) *no\_limit*

```

quotas:
  method: uniform
  limit: 100
  fullquota_content: |
    # Thank you ...
    Unfortunately, you ...

```

(b) *uniform*

```

quotas:
  method: custom
  limit: source/quotas.csv
  fullquota_content: |
    # Thank you ...
    Unfortunately, you ...

```

(c) *custom*

Figure 5: YAML configurations for the three quota modes.

PixelSurvey supports three quota management methods that regulate how respondents are admitted based on their screening responses. The *no\_limit* method imposes no restrictions, allowing all participants to proceed regardless of their answers. The *uniform* method enforces balanced sampling by limiting the number of respondents for each combination of screening responses to a predefined threshold (i.e., `limit` field). The *custom* method offers the highest flexibility: researchers can define quotas externally for any combination of responses, enabling complex sampling strategies such as stratified or quota-based recruitment. If a participant does not meet the eligibility criteria or if the quota for their response category has already been filled, PixelSurvey automatically terminates the survey for that participant and displays a customizable message explaining the situation (i.e., `fullquota_content` field). Table 1 illustrates an example of a custom quota file for two screening questions with two alternatives each, where the `quota` column defines the maximum number of respondents per bin.

Table 1: Extract of a custom quota file (`quotas.csv`) for two screening questions. Each row specifies the maximum number of respondents allowed for a given combination of age and cycling frequency.

| sq1 | age   | label | sq2                    | cycling | label | quota |
|-----|-------|-------|------------------------|---------|-------|-------|
|     | young |       | less than once a month |         |       | 100   |
|     | young |       | more than once a month |         |       | 200   |
|     | adult |       | less than once a month |         |       | 300   |
|     | adult |       | more than once a month |         |       | 500   |

### 3.3.5 Body/instructions

The Instructions page is the mandatory first page of the Body section and serves as the transition between the on-boarding phase and the main research activities. The YAML definition of the Instructions page follows the same structure as the Home and Consent pages, with a single property, `content`, written in Markdown format. This is YAML snippet with a consent page example:

```
instructions:
  content: |
    ## Study Instructions
    Please read these ...
```

### 3.3.6 Body/activity

After the Instructions page, the Body section consists of one or more activities that represent the core of the survey. Each activity is declared in the YAML recipe as a separate page (e.g., `activity_1`, `activity_2`, ...), which specifies the parameters that control its behavior. PixelSurvey allows two types of activities: questionnaires and experiments. Surveys can freely combine these activities in any order to meet the requirements of a study design. The following snippet illustrates an example with two activities, the first a questionnaire and the second an experiment:

```
activity_1:
  order: 1
  type: questionnaire
  persistent_instructions: |
    ## Sociodemographic questions
    Please answer the following questions ...

  questions:
    - id: 1
      ...
    - id: 2
      ...

activity_2:
  order: 2
  type: experiment
  experiment_type: ...
  persistent_instructions: |
    ## Experimental activity
    You will see pairs of ...

  task:
    ...

  settings:
    ...
```

At a general level, each activity definition includes two common properties: `order`, which establishes the sequence in which activities are presented, and `type`, which specifies whether the activity is a *questionnaire* or an *experiment*. Beyond these shared fields, each type of activity has its own structure. Questionnaires contain a single additional field, `questions`, analogous to the Screening page, where a list of multiple-choice or open-text questions can be defined. Experiments, in contrast, always require the field `experiment_type`, which specifies the type of experimental design to be implemented. They also include two additional blocks: `task`, which describes the structure of the experimental stimuli, and `settings`, which define parameters such as randomization or the number of tasks per respondent. The detailed YAML structures for questionnaires, questions, and each experiment type are described in the following subsections.

### 3.3.7 Closure/end

The Closure section concludes the survey with a single page, the `end` page. The YAML definition contains only a `content` property written in Markdown format.

```
end:
  content: |
    ## Thank you!
    We appreciate your time and contribution to this study.
```

### 3.4 Survey components

Activities in PixelSurvey are composed of smaller components that define their content and behavior. Questionnaires are built from a list individual questions, while experiments are defined by specific paradigms that organize tasks and stimuli. These components are encoded in the YAML recipe through structured fields, making them both transparent and extensible. In this section, we describe the currently available components, namely questions for questionnaires and three built-in experiment types. While only a limited set of components is included in the current release, the modular design of PixelSurvey enables the community to extend this library with additional question or experimental types.

#### 3.4.1 Questions for questionnaires

Questions are the fundamental elements of questionnaire activities. Each questionnaire is composed of a list of questions, with each entry in YAML corresponding to a single item. A question element always requires `id`, `type`, `order`, and `question`. Depending on the type, additional properties specify the available responses or constraints. At present, PixelSurvey supports two question types: `multiple-choice` and `open-text`.

**Multiple-choice questions** Multiple-choice questions allow participants to select one predefined alternative from a set of options. In PixelSurvey, these are rendered as drop-down menus, but they can also be adapted to other interface styles such as radio buttons or check boxes with community-contributions. Multiple-choice questions can appear either in the Screening page or in a Questionnaire activity, always defined under the `questions` field. This is a YAML example for a multiple-choice question:

```
- id: 1
  type: multiple_choice
  order: 1
  question: "What is your age group?"
  alternatives:
    - "18-25"
    - "26-35"
    - "36-45"
    - "46+"
  required: true
  variable_name: age_group
```

In this format, the `question` field specifies the text presented to participants, and `alternatives` lists the set of available response options. The `required` field specifies whether an answer must be provided before proceeding; it must be set to `true` in Screening pages but can be freely defined in questionnaires. Finally, the `variable_name` field defines the identifier under which responses are stored in the database, ensuring that data can be consistently retrieved and analyzed.

**Open-text questions** Open-text questions provide participants with a free-form input field to enter qualitative responses. This type is particularly useful for capturing opinions, justifications, or information that cannot be predefined as a set of alternatives. Unlike multiple-choice items, open-text questions do not include additional response fields and can only be used in questionnaire activities, not in the Screening page. The YAML definition for an open-text question is shown below:

```
- id: 2
  type: open_text
  order: 2
  question: "What factors influence your choice of transport mode?"
  required: false
  variable_name: transport_factors
```

Multiple-choice and open-text are basic forms of questions useful for most survey designs. The system, however, is extensible: additional formats—such as sliders, ranking tasks, Likert scales, or image-based inputs—can be integrated in future versions through community contributions.

### 3.4.2 Experiment types

Experiments in PixelSurvey are designed to implement established paradigms in behavioral research. They consist of structured tasks where participants evaluate or choose between stimuli under controlled conditions. Each experiment includes a `task` block, describing what participants see and how they should respond, and a `settings` block, which defines how tasks are generated (e.g., randomization, number of repetitions). PixelSurvey currently implements three experiment types: Stated Choice, Similarity Judgment, and Likert Scale.

The selection of these three experiment types for the initial release of PixelSurvey is motivated by their relevance in behavioral research and their methodological complementarity among them. Stated Choice represents a dominant paradigm in transportation, marketing, and economics for analyzing trade-offs and estimating preferences. Similarity Judgment, in turn, captures perceptual organization and comparative reasoning, and has a long tradition in psychology and cognitive science (van Cranenburgh & Garrido-Valenzuela, 2025; Garrido-Valenzuela et al., 2025). Both paradigms are particularly well-suited to image-based experiments, where visual stimuli form a central part of the evaluation. Finally, the Likert Scale experiment offers a flexible paradigm for obtaining repeated evaluations of images on key dimensions, which can also complement the data generated by the other two experiments. The following subsections describe each experiment type and its implementation in PixelSurvey.

**Stated Choice (SC) experiment** Stated choice (SC) experiments ask respondents to select one option among two or more alternatives defined by attributes Louviere et al. (2000). They are widely applied across fields such as transportation, marketing, environmental and health economics, and consumer research. Their primary purpose is to generate data for estimating discrete choice models Train (2009), which provide insights into economic behavior by quantifying trade-offs and identifying the relative importance of different attributes. In doing so, SC experiments allow researchers to infer measures such as willingness-to-pay.

PixelSurvey implements a binary SC experiment to trade off between two alternatives. One attribute of the alternatives can be an image. This makes it possible to combine traditional textual or numerical attributes (e.g., price, size, travel time) with visual information (e.g., façade design, landscape features) van Cranenburgh & Garrido-Valenzuela (2025). The following YAML snippet describes an SC experiment, where `experiment_type` is set to `stated_choice`:

```
activity_1:
  order: 1
  type: experiment
  experiment_type: stated_choice
  persistent_instructions: |
    ## Activity 1: Choosing your right house
    In the following ...

  task:
    query: "Which option do you prefer?"
    instance: "House"
    n_alternatives: 2
    attributes:
      - label: "Facade"
        type: image
        unit: "url"
      - label: "Price"
        type: standard
        unit: "€"
      - label: "Rooms"
        type: standard
        unit: ""

  settings:
    ...
```

The YAML example defines a stated choice experiment with two alternatives. The `task` block specifies the essential elements of the experimental task. The `query` field provides the question presented to participants, the `instance` field labels the type of object being compared (in this case, “House”), and the `n_alternatives` field indicates the number of options shown per task. At present, PixelSurvey supports choice sets with only two alternatives. Attributes for each alternative

are then listed individually. Each attribute includes a **label**, which is the textual representation of the attribute; a **type**, which can be *image* for visual attributes or *standard* for numerical or textual ones; and a **unit**, which is set to *url* for images but can otherwise be freely defined. In this example, the facade of the house is represented as an image attribute, while price and number of rooms are defined as standard numerical or textual attributes. Figure 6 shows the layout that this YAML SC configuration generates.

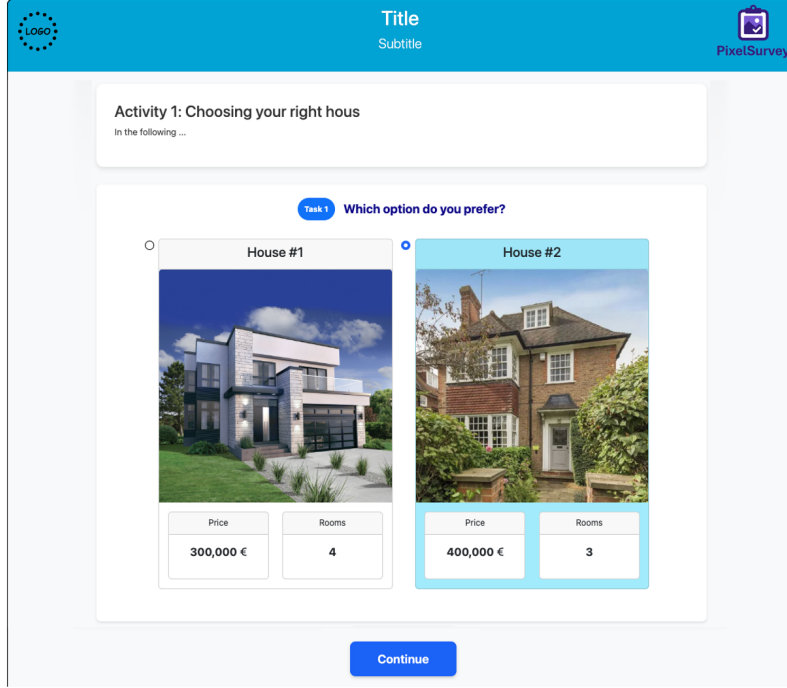


Figure 6: Layout generated with the SC experiment YAML presented

Finally, the **settings** block controls how choice sets are generated and presented to participants. Currently, two different settings modes can be used for all experiments: *custom* or *random*. Figure 7 shows the YAML snippets for both modes.

```
settings:
  tasks_per_respondent: 8
  experimental_design_mode: custom
  custom_design: source/custom_design.csv
```

(a) *custom*

```
settings:
  tasks_per_respondent: 5
  experimental_design_mode: random
  number_of_sets: 100
  attributes_values: source/att_values.csv
```

(b) *random*

Figure 7: YAML configurations for the two settings modes in SC experiments.

The **settings** block determines how experimental tasks are generated and assigned to respondents. The field **tasks\_per\_respondent** specifies the number of choice tasks that each participant must complete. The **experimental\_design\_mode** can be set to either *custom* or *random*. In the custom mode, the researcher supplies a pre-specified experimental design through an external CSV file (**custom\_design**), allowing full control over the combinations of attribute levels shown to respondents. Table 2 illustrates the content of a custom design file corresponding to the YAML example. Each row represents a task with a unique **task\_id**, which is grouped under a **set\_id**. All tasks with the same **set\_id** are completed by the same respondent. Each column specifies the attributes of the alternatives to be presented.

In the random mode, attribute values are drawn randomly from a predefined CSV file (**attributes\_values**). The number of generated sets is defined by the field **number\_of\_sets**. Table 3 shows an example of an attributes values file used to generate random choice sets. The **attribute\_name** column specifies the attribute listed in **task/attributes**, and the **attribute\_value** column enumerates all possible values used to generate random combinations.

Table 2: Excerpt of a `custom_design` file used for full specification of SC tasks.

| task | id | set | id | alt1                | att1 | facade | alt1 | att2   | price | alt1 | att3 | rooms | alt2 | att1 | ... |
|------|----|-----|----|---------------------|------|--------|------|--------|-------|------|------|-------|------|------|-----|
| 1    |    | 1   |    | http://.../img8.png |      |        |      | 100000 |       |      | 2    |       |      |      | ... |
| 2    |    | 1   |    | http://.../img5.png |      |        |      | 100000 |       |      | 1    |       |      |      | ... |
| 3    |    | 1   |    | http://.../img3.png |      |        |      | 300000 |       |      | 2    |       |      |      | ... |

Table 3: Excerpt of an `attributes_values` CSV file used for random generation of SC tasks.

| attribute  | name | attribute               | value    |
|------------|------|-------------------------|----------|
| facade_url |      | http://www.example.com/ | img1.png |
| facade_url |      | http://www.example.com/ | img2.png |
| facade_url |      | http://www.example.com/ | img3.png |
| price      |      | 100000                  |          |
| price      |      | 200000                  |          |
| price      |      | 300000                  |          |
| rooms      |      | 1                       |          |
| rooms      |      | 2                       |          |

**Similarity Judgment (SJ) experiment** Similarity judgment (SJ) experiments ask respondents to evaluate the perceptual similarity or dissimilarity among instances. A common task is the "odd-one-out", where participants identify which item in a triplet is most different compared to the other two. These designs are widely used in psychology and cognitive science to study perceptual organization, category formation, and conceptual reasoning Shepard (1980). More recently, they have been employed in computational cognitive science and machine learning to collect large-scale perceptual data and build similarity-based embeddings (Hebart et al., 2020; Garrido-Valenzuela et al., 2025). Their main purpose is to elicit structured judgments that reflect how humans perceive and organize stimuli, which can then be used to validate theories of perception, construct cognitive models, or align computational representations with human intuitions.

PixelSurvey implements an SJ experiment that compares sets of instances, where each instance is composed of one or more images. The following YAML snippet defines an SJ experiment, where `experiment_type` is set to `similarity_judgment`:

```
activity_2:
  order: 2
  type: experiment
  experiment_type: similarity_judgment
  persistent_instructions: |
    ## Activity 2: Which is the odd...
    In the following ...

  task:
    query: "Which house interior is the odd one out?"
    instance: "House interior"
    images_per_instance: 3

  settings:
    ...
```

In this example, each instance corresponds to a "House interior". The `task` block specifies the `query` field, which defines the question presented to participants; the `instance` field, which labels the type of object being compared; and the `images_per_instance` field, which indicates how many images are used to represent each instance (three in this case). Figure 8 shows the layout that this YAML SJ configuration generates.

The `settings` block is analogous to the SC experiment and controls how task sets are generated and presented. Figure 9 shows the YAML snippets for `custom` and `random` modes.

In `custom` mode, the researcher provides a pre-specified experimental design through an external CSV file (`custom_design`), formatted similarly to SC but with columns that directly list the images associated with each instance. In `random` mode, the key difference with SC that an `instances` CSV file is required instead of an `attributes_values` file. Each row in the `instances` file defines a complete instance, including an identifier and the corresponding image URLs.

**Likert Scale (LS) experiment** Likert scale (LS) experiments ask participants to evaluate one or more dimensions of an instance on a predefined ordinal rating scale (e.g., from 1 to 5) (Likert,

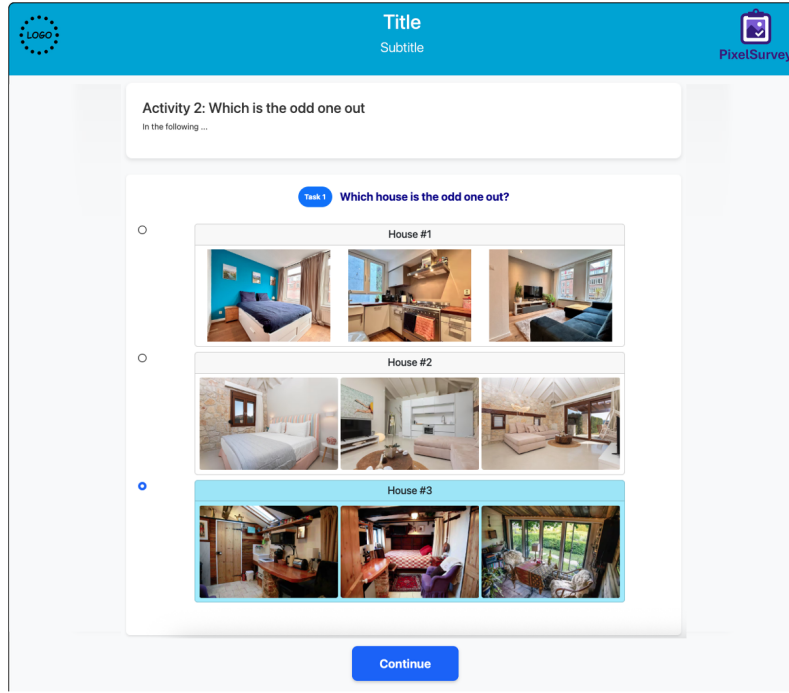


Figure 8: Layout generated with the SJ experiment YAML presented

```
settings:
  tasks_per_respondent: 8
  experimental_design_mode: custom
  custom_design: source/custom_design.csv
```

(a) *custom*

```
settings:
  tasks_per_respondent: 5
  experimental_design_mode: random
  number_of_sets: 100
  instances: source/sj_instances.csv
```

(b) *random*

Figure 9: YAML configurations for the two settings modes in SJ experiments.

1932). Likert scaling has become one of the most widely used tools for measuring attitudes, perceptions, and evaluations across disciplines such as psychology, sociology, education, marketing, and health research (?). Its primary strength lies in capturing subjective judgments in a standardized, repeatable way that facilitates both descriptive analysis and the construction of latent constructs (e.g., attitudes or satisfaction indices). Unlike a traditional Likert-style survey question, which can be potentially developed as a single item in a questionnaire, the LS "experiment" is designed as a structured task that can be repeated across multiple instances and conditions.

Currently, PixelSurvey implements LS as experiments, not as a single-question format. This format allows researchers to systematically measure perceptions, attitudes, or evaluations under controlled designs. The following YAML snippet defines an LS experiment, where `experiment_type` is set to `likert_scale`:



```

activity_3:
  order: 3
  type: experiment
  experiment_type: likert_scale
  persistent_instructions: |
    ## Activity 3: Rating task
    Please rate the following criteria ...

  task:
    query: "Rate the following dimension about the house"
    instance: "House"
    likert_scale: 5
    evaluations:
      - label: "Safety"
      - label: "Aesthetics"

  settings:
    ...

```

In this example, the **task** block defines the rating activity. The **query** field specifies the statement or prompt shown to participants, **instance** labels the type of object being evaluated, **likert\_scale** sets the number of points on the rating scale, and **evaluations** lists the specific dimensions to be rated. Figure 10 shows the layout that this YAML LS configuration generates.

The screenshot shows a web application interface for a rating task. At the top, there is a blue header bar with a logo on the left, the text 'Title' and 'Subtitle' in the center, and a 'PixelSurvey' logo on the right. Below the header, the main content area has a light blue background. A white box contains the title 'Property Rating Task - Random Design' and a subtitle 'You will rate properties on different characteristics using a 5-point scale.' Below this, a task instruction 'Task 1 Rate the following dimension about the house' is shown. A central image of a house is displayed. Below the image, there are two horizontal rating scales. The first scale is for 'Safety' and the second is for 'Beauty'. Each scale has a 5-point rating system with markers at 1, 2, 3, 4, and 5. A 'Continue' button is located at the bottom of the task area.

Figure 10: Layout generated with the LS experiment YAML presented

The **settings** block follows the same structure as in the SJ experiments. In *custom* mode, a full design is provided through a **custom\_design** CSV file, where each row corresponds to a task and set, specifying the instance to be shown and rated. In *random* mode, tasks are generated from instances listed in an external CSV file (**instances**), where each row corresponds to a single instance, which is represented by an image URL.

### 3.5 PixelSurvey Output

Once the YAML recipe and source files are provided to **PixelSurveyCore**, the framework automatically generates a dedicated **Survey** folder. This folder contains all the elements required to execute the survey as a web application and to store participant responses. In other words, the output of PixelSurvey is a web app with an embedded database for managing experimental data and responses. Figure 11 shows the full folder structure for an output survey.



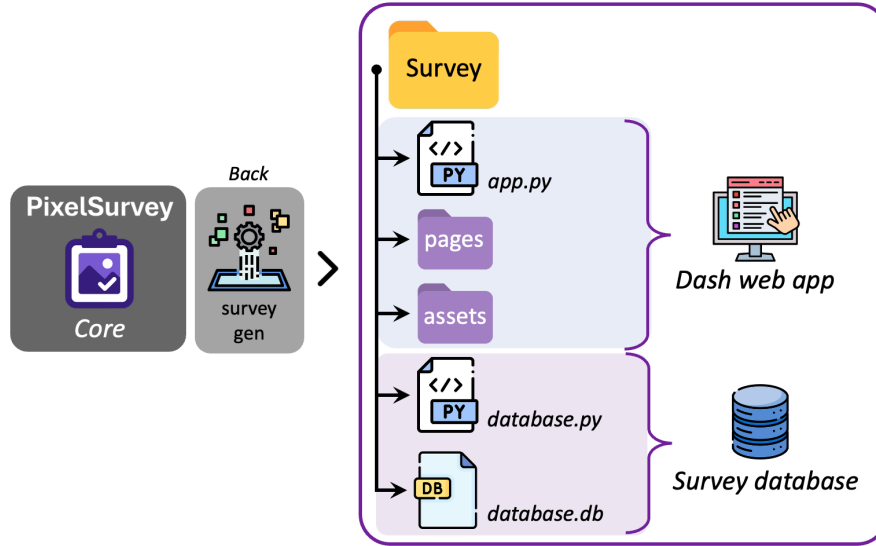


Figure 11: PixelSurvey output: the framework generates a **Survey** folder containing a Dash web app (top) and a dedicated survey database (bottom).

### 3.5.1 Dash web application

The first component of the output is a Dash-based web application coded in Python, which participants interact with to complete the survey. This application is defined by a set of automatically generated files:

- **app.py**, which provides the entry point of the application.
- **pages/**, a folder containing individual pages of the survey (e.g., screening, and the different activities).
- **assets/**, which contains static resources such as logos or stylesheets.

These files form a self-contained web service that can be directly deployed to collect responses. Because all pages are created from the YAML recipe, the generated app exactly reflects the survey design specified by the researcher, ensuring reproducibility and transparency.

### 3.5.2 Survey database

In parallel, PixelSurvey automatically creates a dedicated database to manage experimental tasks and responses. The database is implemented in SQLite format and is stored in the **Survey** folder as **database.db**, with its manager defined **database.py**. The *db* file stores the different tables for generating the experiments and collecting responses, and the *py* file implement the methods for interaction between the web app and the database. This means that researchers do not need to manually configure or maintain the database; instead, they can directly query it to analyze collected data. The internal structure of the database tables is detailed in the following subsections.

**Quotas table** The quotas table records the distribution of respondents across different screening categories and monitors quota fulfillment. It mirrors the structure defined in the screening page (see Table 1), with an additional column that tracks how many participants from each combination of screening responses have already completed the survey. This allows the system to enforce the quota rules in real time as new participants attempt to enter the study.

**Experimental task table** The experimental task table defines the pool of tasks included in each experimental activity. A separate table is created for every experiment declared in the YAML recipe. Each table begins with a unique **task\_id** column, followed by the fields required by the corresponding experiment type. For example, in stated choice experiments, **task\_id** is followed by the attribute values of two alternatives that together define a single choice task.

**Task set table** The task set table organizes experimental tasks into sequences that are assigned to individual respondents. Each row corresponds to a unique `set_id`, which groups together the `task_ids` that a participant must complete. For example, if a survey contains two experimental activities—one with three tasks and another with four—each row of the Task set table will include a `set_id` and seven corresponding `task_ids`, three from the first activity and four from the second.

**Response table** The response table stores all participant responses collected during the survey. Each row corresponds to a `respondent_id` and includes answers to screening questions, questionnaire individual questions, and experimental tasks. Each record links the response to the corresponding participant, activity, and task, ensuring that the full trajectory of individual responses can be reconstructed.

**Timestamp table** The timestamp table logs the precise moment, in microseconds, when participants click the navigation button to move between pages. This provides detailed information about response times at the page level (i.e., task level), supporting analyses of task duration and participant engagement.

## 4 USAGE

This section outlines the typical workflow for running a study with PixelSurveyCore: accessing PixelSurveyCore, preparing a survey recipe, generating the application, deploying it, collecting data, and exporting responses.

### 4.1 Step 1: Accessing PixelSurveyCore

PixelSurveyCore is developed in Python and builds survey applications on top of the Plotly Dash framework. The source code is openly available on GitHub and can be cloned or downloaded directly. Once installed, researchers can generate surveys by providing a recipe (YAML file and source folder) as input.

In addition to the command-line version, a lightweight graphical user interface (GUI) is available. In this variant, users can upload a compressed survey recipe (ZIP file), and the platform automatically compiles and returns the corresponding web application. This lowers the entry barrier for non-technical users.

### 4.2 Step 2: Creating a survey recipe

The second step is to define the survey recipe. As described in the previous section, this consists of a YAML file and a source folder. The YAML file encodes the structure of the survey, including its sections, activities, and settings. The source folder contains all supporting files referenced in the YAML, such as logos, optional quota definitions, and experimental designs in CSV format.

### 4.3 Step 3: Generating the web application

Once the recipe is prepared, the next step is to generate the survey web application. In the command-line version of PixelSurveyCore, this is done by running the `survey_gen` command that compiles the YAML file and source folder into a ready-to-use web application. Internally, the system processes the recipe, creates the necessary database, assembles the front-end and back-end components of the survey, and generates a Dash App project ready to be deployed.

In the GUI version, users upload the ZIP file containing the recipe, and the platform compiles it. At this point, users can either directly download the generated Dash application for local or institutional deployment, or opt to host it directly on the PixelSurvey server. In both cases, the result is a self-contained survey application ready for immediate use.

#### 4.4 Step 4: Deployment and data collection

Once the app is generated, the survey application can be deployed. Researchers have two options: they may download the Dash application and run it locally or on institutional servers, or they may choose to host it directly on the PixelSurvey platform. Hosting on the platform simplifies the deployment and allows surveys to be launched with minimal technical setup, while local deployment provides full control.

There are some trade-offs between local and PixelSurvey-hosted deployment. Hosting locally or on institutional servers gives researchers full control over data storage, access, and compliance with institutional or legal requirements (e.g., GDPR). This approach is generally recommended for large-scale studies, sensitive data, or projects that require strict security policies. By contrast, hosting on the PixelSurvey server lowers technical barriers and simplifies deployment, making it suitable for pilots, classroom use, or small/medium-scale studies. However, this option requires trust in the platform’s infrastructure for handling and storing data, and may be less suited to sensitive or large-scale deployments. Researchers should evaluate their technical resources, ethical obligations, and study requirements before deciding on a hosting option.

For local or institutional deployment, the GitHub repository includes a ready-to-use `docker-compose` recipe that streamlines the process of setting up the server environment. Docker is a lightweight containerization system that packages software together with all its dependencies. This ensures that applications run reliably on the same context. Running the provided `docker-compose` file automatically configures the required services, offering a straightforward way to get the survey system operational without additional manual setup.

Researchers with little or no experience in server administration can request assistance from their institutional IT departments for deployment. In practice, the requirements are minimal: the IT team only needs to set up a server with web ports 80 (HTTP) and 443 (HTTPS) open, install Docker, and run the provided `docker-compose` file together with the output survey folder. This setup automatically builds the PixelSurvey application and database without requiring manual configuration. In future releases, PixelSurvey aims to provide a simplified deployment script that prepares the server environment even more transparently, further easing the workload for IT staff and lowering the barrier for researchers to run their own studies.

During data collection, all participant interactions are stored automatically in the survey’s SQLite database where the survey application is running.

#### 4.5 Step 5: Exporting and accessing data

After data collection, responses are stored in the SQLite database automatically generated with each survey instance. The database contains all relevant tables, including screening responses, questionnaires, experimental tasks, and timestamps. Because it is implemented in SQLite, the file can be easily accessed using a wide range of tools. For example, researchers can open the database directly in Python using `pandas` or in R through packages such as `RSQLite`. Alternatively, any standard SQLite database manager can be used for inspection. Data can be queried directly from the database or exported to CSV format for further statistical analysis, modeling, visualization, or integration with external workflows.

## 5 OPEN-SCIENCE AND COMMUNITY CONTRIBUTIONS

PixelSurvey has been created as a survey tool that embeds open-science principles into its core. Its architecture emphasizes transparency, reproducibility, and collaborative extension, making survey designs easier to share, replicate, and build upon. The subsections below describe how this philosophy is implemented and how researchers can contribute to the framework’s continued development.

### 5.1 Open-science philosophy

PixelSurvey has been designed as a tool which embraces and advances open science. Its reliance on human-readable YAML recipe files ensures that survey designs are easy to create, and straight-

forward to share. With PixelSurvey it is easy to share survey logic, structure, and stimuli considered. Additionally, its framework allows transparency and reproducibility, as other researchers can directly re-run, adapt, or extend a study simply by re-using the original recipe file.

PixelSurvey incorporates explicit versioning practices to ensure long-term reproducibility. Each YAML recipe includes a version number key, which instructs the PixelSurveyCore compiler to interpret the file with the appropriate specification. This ensures that older YAML recipes remain functional even as the platform evolves, making them durable scientific artifacts comparable to datasets, experimental designs, or analysis scripts. Recipes can therefore be archived, cited, and re-used alongside publications, supporting both replication and cumulative research.

## 5.2 Community contributions

The open-source and modular design of PixelSurvey encourages community contributions that extend its functionality. Contributions can take the form of reusable components. This version of PixelSurvey allows the contribution of additional questionnaire item types or entirely new experimental paradigms. Integration into the framework occurs through pull requests to the PixelSurveyCore GitHub repository, ensuring transparent review and collaborative improvement. To contribute a new component, developers are asked to provide the following elements:

- **YAML definition:** A snippet specifying the configuration of the component, following the conventions illustrated for existing question types (e.g., multiple-choice, open-text) and experiments (e.g., stated choice, similarity judgment, Likert scale). The YAML must declare all required structural elements.
- **Parser file:** A Python class that maps the YAML configuration into internal variables accessible within the framework. This file ensures that the declared keys and values are correctly processed by the system.
- **Front-end component:** A Python class implementing the user interface of the component, built on Plotly Dash Dash (2024) and Dash Bootstrap Components Dash Bootstrap Components (2025). All components are organized as `Card` elements, which can contain standard web widgets such as text boxes, radio buttons, or sliders.
- **Source file structure (for experiments):** A clear definition of any external files required to instantiate experimental tasks. These are the CSV files containing attributes, or instances. This guarantees that the platform can correctly load, display, and randomize tasks.

Looking forward, we envision PixelSurvey as a continuously evolving ecosystem. Planned developments include the addition of a graphical user interface (GUI) to lower entry barriers for non-technical users, the integration of further experimental paradigms, and community-driven libraries of survey templates. In this way, PixelSurvey aims not only to support current research needs but also to anticipate future demands by combining open science principles with collaborative software development.

## 6 CONCLUSIONS

PixelSurvey was developed to fill the absence of a flexible, open-source framework capable of supporting experimental designs with image-based stimuli at scale. It enables the creation of survey experiments in which large image datasets can be easily integrated into experimental designs. The framework lowers technical barriers by adopting a modular architecture based on human-readable YAML files. These files also ensure that surveys can be shared, archived, and replicated as scientific artifacts, thereby contributing directly to open-science practices.

The framework is able to create surveys with two question types and three experiment paradigms — i.e. Stated Choice, Similarity Judgment, and Likert Scale. Those experiments cover a wide spectrum of behavioral research needs. Its extensibility, however, ensures that new components can be further expanded over time, whether developed by the core team or contributed by the community. Through its design, PixelSurvey broadens methodological possibilities for image-based studies and provides a robust platform for creating experiments built on textual or numerical inputs.

Looking ahead, the continued development of PixelSurvey will be driven by community engagement and the integration of additional experimental paradigms, user-friendly interfaces, and shared libraries of survey templates. In doing so, the framework aspires to become a long-term resource for researchers across disciplines, fostering cumulative knowledge building and accelerating innovation in the study of human perception, preferences, and decision-making.

## REFERENCES

- Al-Omari, B., Farhat, J., & Ershaid, M. (2022). Conjoint analysis: A research method to study patients’ preferences and personalize care. *Journal of Personalized Medicine*, 12(2), 274.
- Chen, D. L., Schonger, M., & Wickens, C. (2016). otree—an open-source platform for laboratory, online, and field experiments. *Journal of Behavioral and Experimental Finance*, 9, 88–97.
- Dash. (2024). *Dash: A python framework for building analytical web applications*. Retrieved from <https://dash.plotly.com/> (Accessed: YYYY-MM-DD)
- Dash Bootstrap Components. (2025). *dash-bootstrap-components: Bootstrap components for plotly dash*. <https://github.com/dbc-team/dash-bootstrap-components>. (Accessed: 2025-10-03, Apache-2.0 license)
- De Leeuw, J. R. (2015). jspsych: A javascript library for creating behavioral experiments in a web browser. *Behavior research methods*, 47, 1–12.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... others (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*.
- Forster, A. G., & Neugebauer, M. (2024). Factorial survey experiments to predict real-world behavior: A cautionary tale from hiring studies. *Sociological Science*, 11, 886–906.
- Garrido-Valenzuela, F., Cats, O., & van Cranenburgh, S. (2025). From pixels to perceptions: using human similarity judgments to enrich urban space embeddings. *Unpublished*.
- Green, C., & Gerard, K. (2009). Exploring the social value of health-care interventions: a stated preference discrete choice experiment. *Health economics*, 18(8), 951–976.
- Hainmueller, J., Hopkins, D. J., & Yamamoto, T. (2014). Causal inference in conjoint analysis: Understanding multidimensional choices via stated preference experiments. *Political analysis*, 22(1), 1–30.
- Hanemann, M., Loomis, J., & Kanninen, B. (1991). Statistical efficiency of double-bounded dichotomous choice contingent valuation. *American journal of agricultural economics*, 73(4), 1255–1263.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the ieee conference on computer vision and pattern recognition* (pp. 770–778).
- Hebart, M. N., Zheng, C. Y., Pereira, F., & Baker, C. I. (2020). Revealing the multidimensional mental representations of natural objects underlying human similarity judgements. *Nature human behaviour*, 4(11), 1173–1185.
- Henninger, F., Shevchenko, Y., Mertens, U. K., Kieslich, P. J., & Hilbig, B. E. (2022). lab.js: A free, open, online study builder. *Behavior Research Methods*, 54(2), 556–573.
- John, L. K., Loewenstein, G., & Prelec, D. (2012). Measuring the prevalence of questionable research practices with incentives for truth telling. *Psychological science*, 23(5), 524–532.
- Leiner, D. J. (2025). *Sosci survey*. <https://www.soscisurvey.de/>. (Accessed: 2025-10-02)
- Likert, R. (1932). A technique for the measurement of attitudes. *Archives of psychology*.
- LimeSurvey. (2025). *Limesurvey: An open source survey tool*. <https://www.limesurvey.org>. (Accessed: 2025-05-08)

- Liu, Z., Mao, H., Wu, C.-Y., Feichtenhofer, C., Darrell, T., & Xie, S. (2022). A convnet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 11976–11986).
- López Ortega, A., & Radojevic, M. (2025). Visual conjoint vs. text conjoint and the differential discriminatory effect of (visible) social categories. *Political Behavior*, 47(1), 335–353.
- Louviere, J. J., Hensher, D. A., & Swait, J. D. (2000). *Stated choice methods: Analysis and applications*. Cambridge: Cambridge University Press.
- Mariel, P., Hoyos, D., Meyerhoff, J., Czajkowski, M., Dekker, T., Glenk, K., ... others (2021). *Environmental valuation with discrete choice experiments: Guidance on design, implementation and data analysis*. Springer Nature.
- Munafò, M. R., Nosek, B. A., Bishop, D. V., Button, K. S., Chambers, C. D., Percie du Sert, N., ... Ioannidis, J. P. (2017). A manifesto for reproducible science. *Nature human behaviour*, 1(1), 0021.
- Nosek, B. A., Alter, G., Banks, G. C., Borsboom, D., Bowman, S. D., Breckler, S. J., ... others (2015). Promoting an open research culture. *Science*, 348(6242), 1422–1425.
- Ortuzar, J. d. D., Martinez, F. J., & Varela, F. J. (2000). Stated preferences in modelling accessibility. *International Planning Studies*, 5(1), 65–85.
- Paivio, A. (1969). Mental imagery in associative learning and memory. *Psychological review*, 76(3), 241.
- Palma, D., Ortúzar, J. d. D., Rizzi, L., & Casaubon, G. (2018). Modelling consumers’ heterogeneous preferences: a case study with Chilean wine consumers. *Australian Journal of Grape and Wine Research*, 24(1), 51–61.
- Peirce, J., Gray, J. R., Simpson, S., MacAskill, M., Höchenberger, R., Sogo, H., ... Lindeløv, J. K. (2019). Psychopy2: Experiments in behavior made easy. *Behavior research methods*, 51, 195–203.
- PIHappiness. (2025). *What are image surveys?* <https://www.pihappiness.com/what-are-image-surveys/>. (Accessed: 2025-05-08)
- Pinker, S. (1990). A theory of graph comprehension.
- Qualtrics. (2025). *Qualtrics survey platform*. <https://www.qualtrics.com>. (Accessed: 2025-05-08)
- Questback GmbH. (2025). *Unipark / efs survey*. <https://www.unipark.com/>. (Accessed: 2025-10-02)
- Rizzi, L. I., Limonado, J. P., & Steimetz, S. S. (2012). The impact of traffic images on travel time valuation in stated-preference choice experiments. *Transportmetrica*, 8(6), 427–442.
- Rose, J. M., & Bliemer, M. C. (2014). Stated choice experimental design theory: the who, the what and the why. In *Handbook of choice modelling* (pp. 152–177). Edward Elgar Publishing.
- Salm, J. P., Bočkarjova, M., Botzen, W., & Runhaar, H. (2023). Citizens’ preferences and valuation of urban nature: Insights from two choice experiments. *Ecological economics*, 208, 107797.
- Schwitter, N. (2025). Using artificial intelligence to generate visual vignettes in factorial survey experiments.
- Shepard, R. N. (1980). Multidimensional scaling, tree-fitting, and clustering. *Science*, 210(4468), 390–398.
- Steiner, P. M., Atzmüller, C., & Su, D. (2016). Designing valid and reliable vignette experiments for survey research: A case study on the fair gender income gap. *Journal of Methods and Measurement in the Social Sciences*, 7(2), 52–94.
- SurveyEngine. (2025). *Surveyengine*. <https://surveyengine.com>. (Accessed: 2025-05-08)

- SurveyLegend. (2025). *Beautiful image & picture-based surveys*. <https://www.surveylegend.com/user-guide/beautiful-image-picture-based-surveys/>. (Accessed: 2025-05-08)
- SurveyMonkey. (2025). *Surveymonkey online surveys*. <https://www.surveymonkey.com>. (Accessed: 2025-05-08)
- Train, K. E. (2009). *Discrete choice methods with simulation* (2nd ed.). Cambridge: Cambridge University Press.
- van Cranenburgh, S., & Garrido-Valenzuela, F. (2025). Computer vision-enriched discrete choice models, with an application to residential location choice. *Transportation Research Part A: Policy and Practice*, 192, 104300.
- Vecchiato, A., & Munger, K. (2025). Introducing the visual conjoint, with an application to candidate evaluation on social media. *Journal of Experimental Political Science*, 12(1), 57–71.